

Introdução à Computação Gráfica
Projecções

Adaptação: João Paulo Pereira

António Costa

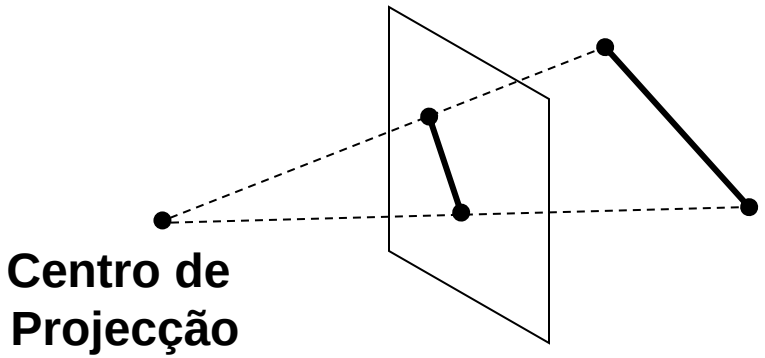
Autoria: Claudio Esperança

Paulo Roma Cavalcanti



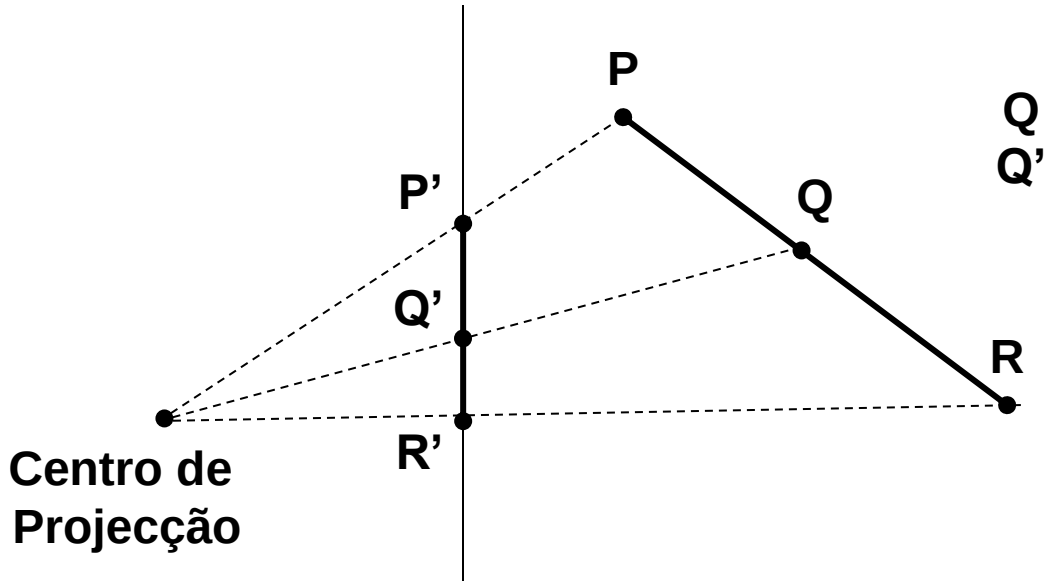
Perspectiva

- É o estudo de transformações projectivas
- O problema consiste em projectar pontos do espaço *d*-dimensional no plano *d*-1 dimensional usando um ponto especial designado por centro de projecção



Transformações Projectivas

- As transformações projectivas transformam rectas em rectas mas não preservam as combinações afim

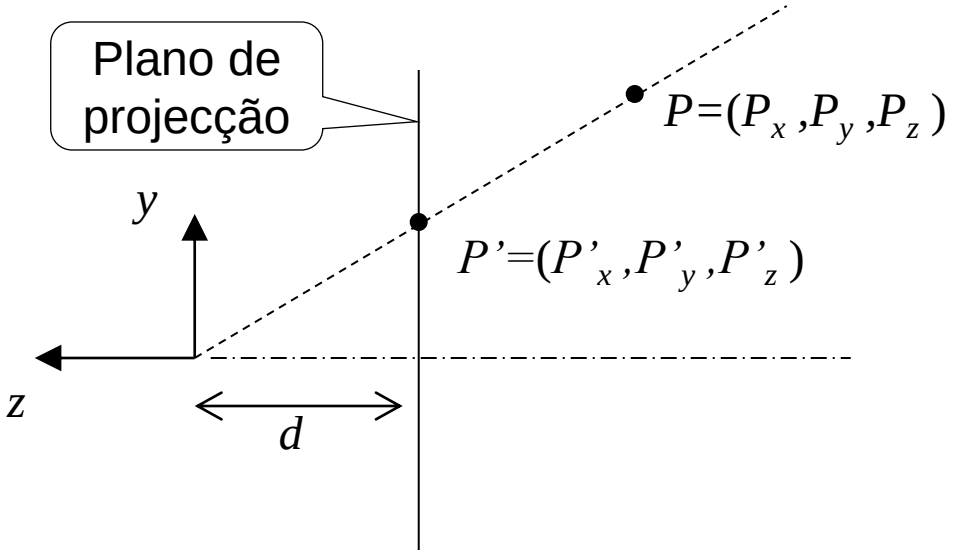


Q é o ponto médio de PR, mas Q' não é o ponto médio de P'R'

Coordenadas homogéneas em espaço projectivo

- Representamos apenas pontos (não vectores)
- Em 2D, um ponto (x,y) será representado em c.h. pela matriz-coluna $[x \cdot w \quad y \cdot w \quad w]^T$, para $w \neq 0$
  - Assim, o ponto (4,3) pode ser representado por $[8\ 6\ 2]^T$, $[12\ 9\ 3]^T$, $[-4\ -3\ -1]^T$, etc.
- Dado um ponto com coordenadas homogéneas $[x \ \ y \ \ w]^T$, a sua representação canónica é dada por $[x/w \ \ y/w \ \ 1]^T$
  Designamos essa operação por *divisão perspectiva*
- Considere os pontos da recta $x=y$: (1,1), (2,2), etc
  - Podemos representá-los em c.h. por $[1\ 1\ 1]^T$, $[1 \ \ 1 \ \ ½]^T$, etc.
  - Claramente, o ponto ideal dessa recta é dado por $[1\ 1\ 0]^T$

Transformações projectivas



- Se o plano de projecção é perpendicular ao eixo *z*, está a uma distância *d* do C.P. (que está na origem) e intercepta o semi-eixo z negativo, então a projecção de um ponto P é dada por

$$P' = \begin{bmatrix} \dfrac{P_x}{-P_z/d} & \dfrac{P_y}{-P_z/d} & -d & 1 \end{bmatrix}^T$$

- Por semelhança de triângulos, vemos que $P_y / -P_z = P'_y / d$

Transformação perspectiva em coordenadas homogéneas

- Não existe matriz 4x4 capaz de realizar tal transformação em espaços euclidianos, mas se assumirmos que o ponto está no espaço projectivo, então

$$P' = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix} \times \begin{bmatrix} P_x \\ P_y \\ P_z \\ 1 \end{bmatrix} = \begin{bmatrix} P_x \\ P_y \\ P_z \\ -P_z/d \end{bmatrix} = \begin{bmatrix} \frac{P_x}{-P_z/d} \\ \frac{P_y}{-P_z/d} \\ -d \\ 1 \end{bmatrix}$$

Perspectiva - Sumário

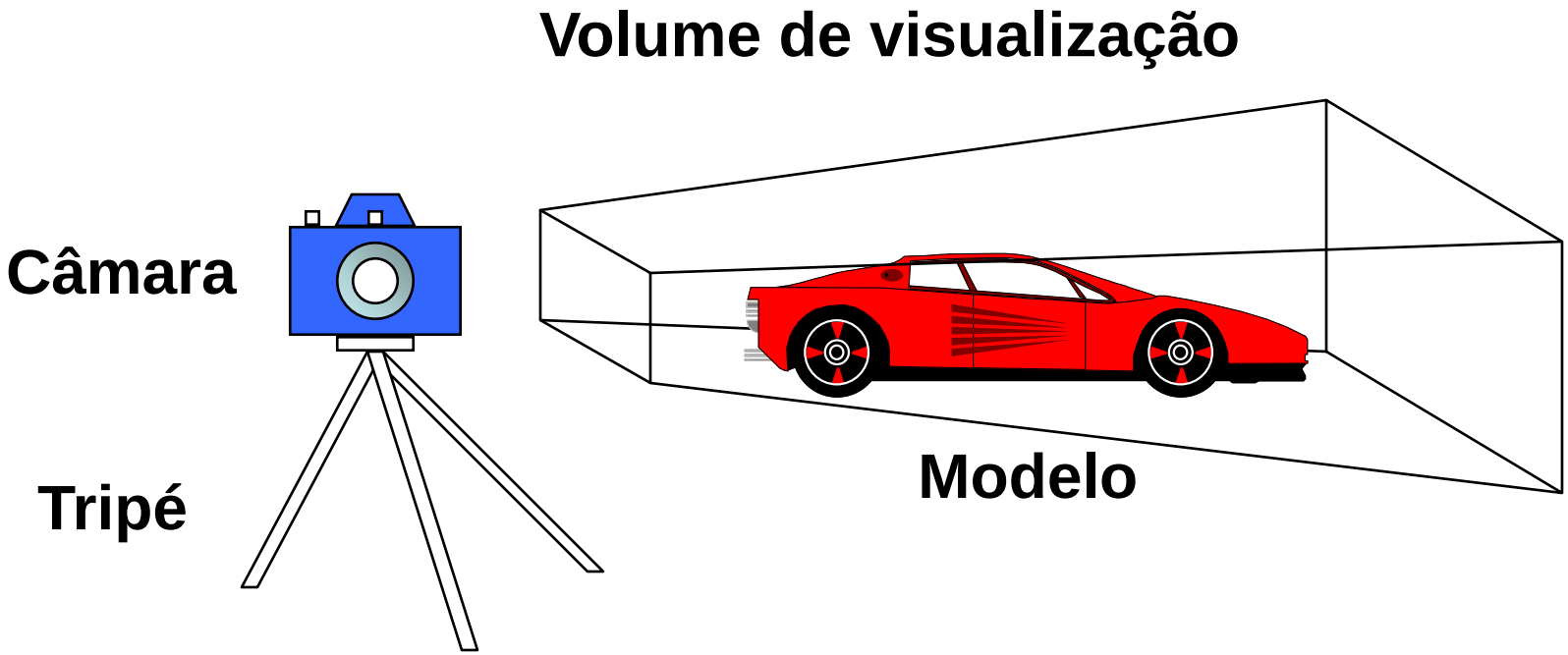
- Para fazer projecção perspectiva de um ponto *P*, seguem-se os seguintes passos
  - *P* é levado do espaço euclidiano para o projectivo
    - Trivial – mesmas coordenadas homogéneas
  - *P* é multiplicado pela matriz de transformação perspectiva resultando em *P'*
  - *P'* é levado novamente ao espaço euclidiano
    - Operação de divisão perspectiva

Projecção genérica

- E se não quisermos que o Centro de Projecção esteja na origem ou se a cena não estiver correctamente posicionada no semi-eixo z negativo?
  - Aplicam-se transformações afins para posicionar todos os elementos correctamente
  - As maneiras pelas quais essas transformações são feitas caracterizam um dado modelo de projecção

Modelo de câmara sintética

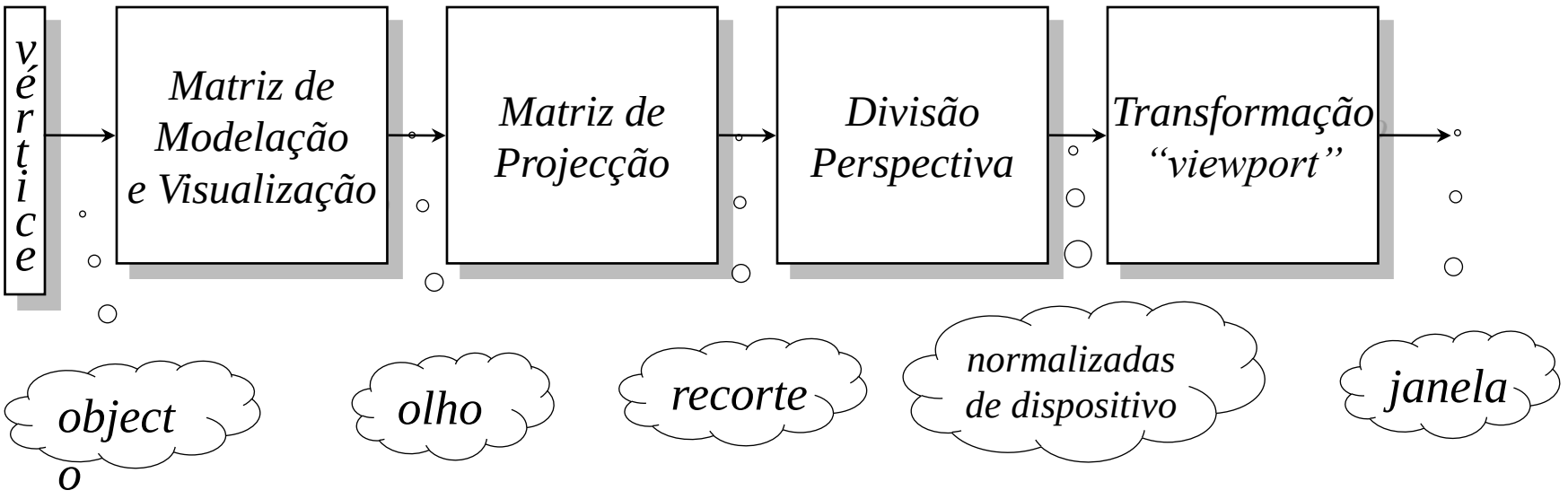
- API utiliza uma analogia comparando visualização 3D com tirar fotografias com uma câmara



Tipos de transformações

- Modelação
  - Mover / deformar os objectos
- Visualização
  - Mover e orientar a câmara
- Projecção
  - Ajuste da lente / objectiva da câmara
- "*Viewport*"
  - Aumentar ou reduzir a fotografia

Pipeline de Transformações



Estado inicial do *pipeline*

- Inicialmente,
  - As matrizes "*modelview*" e "*projection*" são matrizes-identidade
    - Vértices não são transformados e a projecção é paralela ao plano *xy*
    - O mundo visível é restrito ao cubo $-1 \leq x,y,z \leq 1$
  - A transformação "*viewport*" transforma o quadrado $-1 \leq x,y \leq 1$ (em coordenadas normalizadas de dispositivo) na superfície total da janela

Especificação do *viewport*

- Para especificar a área da janela na qual será transformado o quadrado do plano de projecção, utiliza-se

```
renderer.setViewport (x: Integer, y: Integer, width:
  Integer, height: Integer)
renderer.setViewport (vector: Vector4)
```

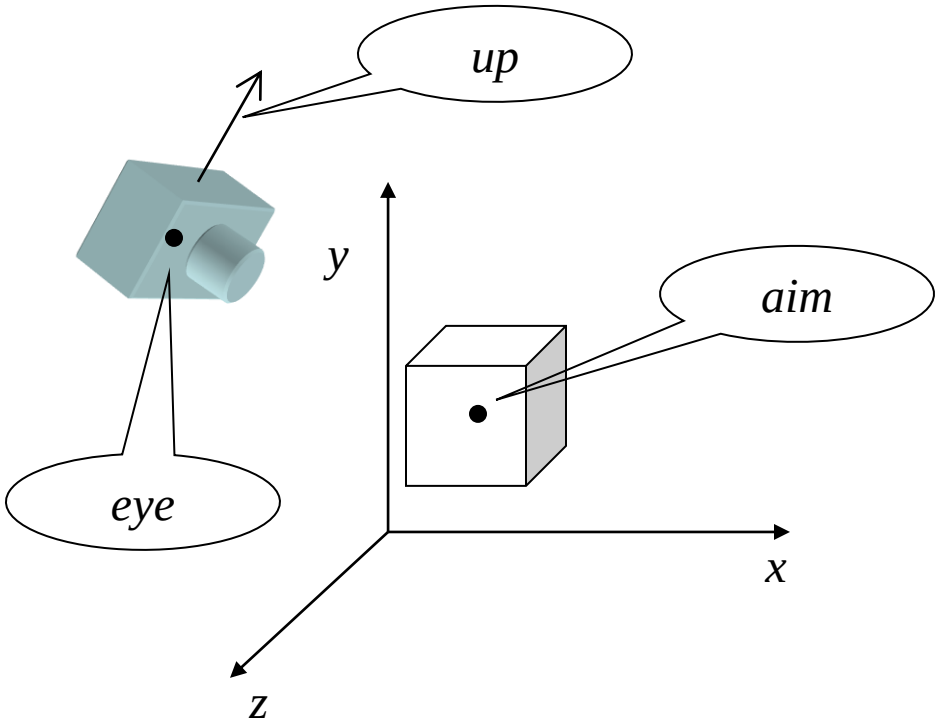
  - (parâmetros em *pixels,* sendo que (0, 0) se refere ao canto inferior esquerdo da janela)
- Normalmente não é necessário modificar, mas é útil para
  - Manter a razão de aspecto da imagem
  - Fazer *zooming* e *panning* sobre a imagem

Transformação de objectos

- Usam-se funções para multiplicar o topo da pilha de matrizes por transformações especificadas por parâmetros
  - **`object.position: Vector3`**
  - **`object.translate*(distance: Float)`**
  - **`object.rotation: Euler`**
  - **`object.rotate* (rad: Float)`**
  - **`object.scale: Vector3`**
- Cuidado: a ordem é importante

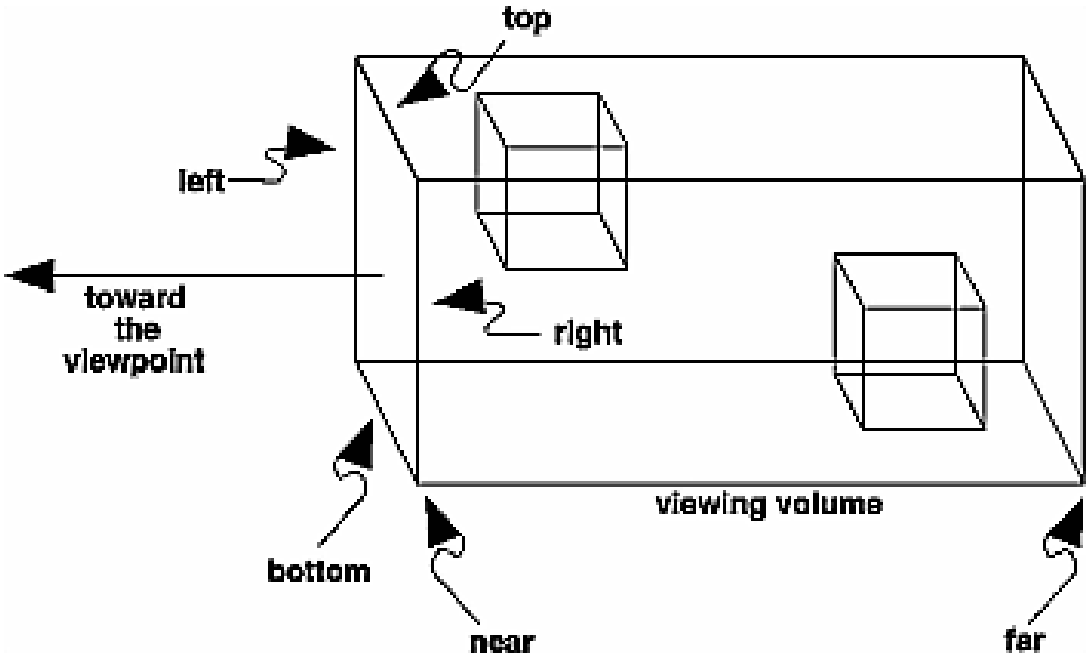
Transformações de visualização

- Duas interpretações:
  - Levam a câmara até a cena que se quer visualizar
  - Levam os objectos da cena até uma câmara estacionária
- `camera.position: Vector3`
- `camera.lookAt(vector: Vector3)`
- `camera.lookAt (x: Float, y: Float, z: Float);`
- `camera.up: Vector3`
  - eye = ponto onde a câmara será posicionada
  - aim = ponto para onde a câmara será apontada
  - up = vector que dá a direção "para cima" da câmara
  - *Cuidado com casos degenerados*



Projecção Paralela

- Volume de visualização especificado com
- `OrthographicCamera(left: Number, right: Number, top: Number, bottom: Number, near: Number, far: Number)`
  - Obs.: near >= 0; far > near



Projecção Perspectiva

- Volume de visualização especificado com `PerspectiveCamera(fov: Number, aspect: Number, near: Number, far: Number)`
  - Obs.: near > 0; far > near

